

Got it. Your feedback is specific and makes sense. Going forward I'll adjust the depth like this:

- **Follow the wording of the topic.** If it says *"at a beginner level"*, I'll stay beginner-friendly and avoid unnecessary internals.
 - **Go deep only where the roadmap expects it.** (For example: Streams, Collections, Multithreading, JVM, Spring, etc.)
 - **Increase coding examples significantly.** Every topic will have:
 - 3–6 simple examples
 - 1 real-world example
 - 1 common mistake example
 - 1 interview-oriented example
 - **Focus on what you'll actually write in Java**, not just theory.
 - **Still include interview questions, tricky questions, summary, and one-liners.**
-

These 6 topics are actually one family

These topics are tightly connected:

```
Functional Interface
  ↓
Lambda Expression
  ↓
Built-in Functional Interfaces
  ↓
Predicate
Function
Consumer
Supplier
  ↓
Method Reference
  ↓
Streams (next topics)
```

I'll teach them in this order because each topic builds on the previous one.

1. Write Lambda Expressions with No Parameter, One Parameter, and Multiple Parameters

After learning Functional Interfaces, the next step is learning **Lambda Expressions**.

A **lambda expression** is simply a **short way of implementing the single abstract method of a Functional Interface**.

Instead of writing an anonymous class, we write a lambda.

General Syntax

```
(parameters) -> {  
    // code  
}
```

Think of it as:

```
(parameters) -> behavior
```

1. No Parameter Lambda

Suppose our interface is

```
@FunctionalInterface  
interface Greeting {  
    void greet();  
}
```

Traditional Way

```
Greeting g = new Greeting() {  
    @Override  
    public void greet() {  
        System.out.println("Hello");  
    }  
};
```

Lambda Way

```
Greeting g = () -> System.out.println("Hello");  
  
g.greet();
```

Output

```
Hello
```

Notice

```
()
```

means

No parameters.

Example 2

```
@FunctionalInterface  
interface StartEngine {  
    void start();  
}  
  
public class Main {  
    public static void main(String[] args) {
```

```
        StartEngine engine = () -> System.out.println("Engine Started");  
        engine.start();  
    }  
}
```

Output

```
Engine Started
```

2. One Parameter Lambda

Interface

```
@FunctionalInterface  
interface Printer {  
    void print(String message);  
}
```

Lambda

```
Printer p = (message) -> System.out.println(message);  
p.print("Java");
```

Output

```
Java
```

Parentheses are Optional (One Parameter)

These two are identical.

```
Printer p = (msg) -> System.out.println(msg);
```

```
Printer p = msg -> System.out.println(msg);
```

Most developers prefer the second form.

Example 2

```
@FunctionalInterface
interface Square {
    int square(int x);
}

Square s = n -> n * n;

System.out.println(s.square(5));
```

Output

```
25
```

3. Multiple Parameters

Interface

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
```

Lambda

```
Calculator calc = (a, b) -> a + b;  
  
System.out.println(calc.add(10, 20));
```

Output

```
30
```

Example 2

```
@FunctionalInterface  
interface MaxNumber {  
    int max(int a, int b);  
}  
  
MaxNumber max = (a, b) -> a > b ? a : b;  
  
System.out.println(max.max(7, 12));
```

Output

```
12
```

Multiple Statements

If the lambda has more than one statement, use braces and return.

```
Calculator calc = (a, b) -> {  
    System.out.println("Adding numbers");  
}
```

```
    return a + b;  
};
```

Real Project Example

Sorting employees

```
employees.sort((e1, e2) ->  
    e1.getSalary() - e2.getSalary());
```

Filtering

```
list.removeIf(x -> x < 10);
```

Thread

```
Thread t = new Thread(() -> {  
    System.out.println("Running...");  
});  
  
t.start();
```

Common Mistakes

Wrong

```
(a,b) =>
```

Java uses

```
->
```

Wrong

```
(a,b) -> {  
    a+b;  
}
```

Needs return.

```
(a,b) -> {  
    return a+b;  
}
```

Wrong

```
() -> return 5;
```

Correct

```
() -> 5
```

or

```
() -> {  
    return 5;  
}
```

Interview Questions

Why are lambda expressions possible?

Because of Functional Interfaces.

Can lambda exist without a Functional Interface?

No.

Can lambda implement two methods?

No.

Why use lambda?

- Less code
 - Better readability
 - Functional programming
 - Stream API
-

One-liner

A lambda expression is a concise way to implement the single abstract method of a Functional Interface.

2. Use Predicate for Boolean Checks

A **Predicate** is a built-in Functional Interface used when you want to **test a condition**.

It always:

- Takes **one input**
- Returns **boolean**

Signature

Predicate

Method

```
boolean test(T t)
```

Think:

```
Input → true/false
```

Example 1

```
Predicate isEven = n -> n % 2 == 0;  
  
System.out.println(isEven.test(8));  
System.out.println(isEven.test(7));
```

Output

```
true  
false
```

Example 2

```
Predicate isEmpty = str -> str.isEmpty();  
  
System.out.println(isEmpty.test(""));  
System.out.println(isEmpty.test("Java"));
```

Output

```
true
```

```
false
```

Example 3

```
Predicate isAdult = age -> age >= 18;  
  
System.out.println(isAdult.test(22));
```

Output

```
true
```

Real Project Example

```
users.stream()  
    .filter(user -> user.isActive())  
    .forEach(System.out::println);
```

The condition inside filter() is a Predicate.

Interview Summary

- Returns boolean
- Method = test()
- Used for filtering and validation

One-liner: A Predicate checks whether a value satisfies a condition.

3. Use Function for Transformation

A **Function** transforms one value into another.

Input → Output

Method

```
R apply(T t)
```

Example 1

```
Function length = str -> str.length();  
System.out.println(length.apply("Java"));
```

Output

```
4
```

Example 2

```
Function square = n -> n * n;  
System.out.println(square.apply(6));
```

Output

```
36
```

Example 3

```
Function upper =  
    s -> s.toUpperCase();  
  
System.out.println(upper.apply("java"));
```

Output

```
JAVA
```

Real Project

```
names.stream()  
    .map(String::toUpperCase)  
    .toList();
```

map() uses a Function.

Interview Summary

- Method = apply()
- Converts one value into another
- Used in map()

One-liner: A Function transforms an input into an output.

4. Use Consumer for Consuming Values Without Return

A **Consumer** takes input but returns **nothing**.

Method

```
void accept(T t)
```

Example 1

```
Consumer print = msg -> System.out.println(msg);  
print.accept("Hello");
```

Example 2

```
Consumer square =  
    n -> System.out.println(n * n);  
square.accept(7);
```

Output

```
49
```

Real Project

```
list.forEach(System.out::println);
```

forEach() uses a Consumer.

Interview Summary

- Input ✓
- Return ✗
- Method = accept()

One-liner: A Consumer performs an action using a value without returning anything.

5. Use Supplier for Lazy Value Creation

A **Supplier** provides a value when requested.

It takes **no input** and returns a value.

Method

```
T get()
```

Example 1

```
Supplier random =  
    () -> Math.random();  
  
System.out.println(random.get());
```

Example 2

```
Supplier message =  
    () -> "Welcome";  
  
System.out.println(message.get());
```

Example 3 (Lazy Creation)

```
Supplier<List> listSupplier =  
    ArrayList::new;  
  
List list = listSupplier.get();
```

The list is created **only when get() is called**, which is why it's called *lazy value creation*.

Interview Summary

- No input
- Returns value
- Method = get()

One-liner: A Supplier creates or supplies a value only when requested.

6. Use Method References for Simple Existing Method Calls

A **Method Reference** is a shorter way to write a lambda **when the lambda only calls an existing method**.

Instead of

```
x -> System.out.println(x)
```

write

```
System.out::println
```

Example 1

```
List names =  
    List.of("Ali", "Ahmed", "Sara");  
  
names.forEach(System.out::println);
```

Example 2

Instead of

```
Function upper =  
    s -> s.toUpperCase();
```

Use

```
Function upper =  
    String::toUpperCase;
```

Example 3

Instead of

```
Supplier> supplier =  
    () -> new ArrayList<>();
```

Use

```
Supplier> supplier =  
    ArrayList::new;
```

This is a **constructor reference**.

Common Mistake

Method references **cannot** be used if you need extra logic.

Wrong:

```
x -> {  
    System.out.println(x);  
    return x * 2;  
}
```

This cannot be replaced by a method reference because there are multiple operations.

Quick Interview Comparison

Interface	Input	Output	Main Method	Typical Use
Predicate	1	boolean	test()	Check conditions
Function	1	1 value	apply()	Transform values
Consumer	1	void	accept()	Perform an action
Supplier	0	1 value	get()	Lazily create/provide values

Overall Interview Questions

1. Why do lambdas require Functional Interfaces?
2. Difference between Predicate and Function?
3. Which interface returns void?
4. Which interface takes no parameters?
5. Why is Supplier called lazy?
6. Difference between a lambda and a method reference?
7. Can every lambda be replaced with a method reference? (No)
8. Which Stream methods commonly use Predicate, Function, and Consumer? (filter, map, and forEach)
9. Which built-in interface would you use for validation? (Predicate)
10. Which one would you use to convert an object to another type? (Function)

This level is what I'd recommend for your internship roadmap: practical, coding-focused, and interview-ready without going deeper than the topics require.